

ANARCHY

AI Platform

Functional Audit

Technical Audit Report — File 3 of 8

10	6.4	3	12+
Features Audited	Avg Feature Score	Broken Features	Missing Features

File	3 of 8 — Functional Audit
Features	Generate, Upscale, Variations, History, Library, Canvas, Billing, Auth, Export, Notifications
Method	Feature-by-feature functional testing analysis
Date	2025 — Confidential Internal Report

Table of Contents

1	Functional Overview & Score Matrix	03
2	Core AI Features	04
2	Generate (Image Generation)	04
1		
2	Upscale	05
2		
2	Variations	05
3		
3	Data & Organization Features	06
3	History	06
1		
3	Library	07
2		
4	Canvas & Workflow Features	08
4	Canvas (Node-Based Workflow)	08
1		
5	Platform Features	09
5	Billing & Credits	09
1		
5	Authentication	10
2		
5	Export	10
3		
5	Notifications	11
4		
6	Expected Bugs & Failure Modes	12
7	Feature Gap Analysis vs VizMaker	13
8	Functional Improvement Roadmap	15

1. Functional Overview & Score Matrix

Complete audit of all 10 major features — working state, bugs, and missing capabilities

The functional audit evaluates each feature on three axes: implementation completeness (what is built vs what was designed), stability (bug frequency and severity), and user experience quality (does it feel professional-grade). The overall functional score of 6.4/10 reflects a platform that works well for basic use cases but has significant gaps in edge case handling and power-user workflows.

Feature	Score	Status	Assessment
Generate	7.0/10	PARTIAL	Core flow works; missing queue, cancel, seed UI for FLUX
Upscale	6.5/10	PARTIAL	Works but no batch mode, no format options, slow feedback
Variations	6.0/10	PARTIAL	Works for FLUX; Nano Banana 2 seed issue causes identical outputs
History	5.5/10	PARTIAL	No pagination, search is slow, no re-use workflow
Library	5.0/10	PARTIAL	Basic exists; missing collections, tagging, smart folders
Canvas	6.5/10	PARTIAL	ReactFlow→LiteGraph migration incomplete; hybrid instability
Billing	6.8/10	PARTIAL	Stripe integration exists; webhook verification incomplete
Authentication	7.5/10	WORKING	Supabase Auth solid; missing MFA, social login limited
Export	6.0/10	PARTIAL	Image + PDF works; DXF pipeline fragile, no batch export
Notifications	7.0/10	WORKING	Toast system works; missing grouping, no notification history

2. Core AI Features

Analysis of the three primary AI generation capabilities

2.1 Generate — Image Generation

■ Generate / Image Generation	7.0/10	PARTIAL
The core generation feature — taking a prompt and architectural image, sending to Replicate (FLUX or Nano Banana 2), and returning a rendered result. This is the most-used feature and its core path works well. However, it lacks the production-grade reliability features (queue, cancel, progress) needed for professional users who run 10+ generations per session.		
Working ✓ ✓ Basic prompt → image generation via FLUX works correctly ✓ img2img (image + prompt) flow functions for Nano Banana 2 ✓ Model switching between FLUX and Nano Banana 2 works ✓ Results correctly injected into canvas as output nodes ✓ Style presets correctly modify generation parameters ✓ Error message shown to user when generation fails	Broken ✗ ✗ No job queue — concurrent generations race and timeout ✗ Generate button not disabled while job is running — double-submit ✗ No cancel button — user must wait for timeout (30-60s) ✗ Seed parameter UI shown for Nano Banana 2 (which ignores seeds) ✗ Progress indicator is fake timer, not real Replicate progress ✗ On network drop mid-generation, result is silently lost	Missing ■ ■ Generation queue with position indicator (e.g., '2nd in queue') ■ Real-time progress from Replicate webhook ■ Cancel/abort running generation ■ Batch generation (run same prompt with 4 different seeds) ■ Prompt history dropdown (recent prompts) ■ Negative prompt field ■ Generation cost preview before submitting

2.2 Upscale

■ Upscale	6.5/10	PARTIAL
Upscales a generated image using a Replicate upscaler model. Currently supports 2x and 4x upscaling via the Real-ESRGAN model. The feature works for its basic use case but lacks batch processing, format control, and quality comparison views.		

Working ✓ ✓ 2x upscale via Real-ESRGAN works correctly ✓ 4x upscale works correctly ✓ Upscaled image correctly replaces or adds alongside original in canvas ✓ Progress shown during upscale (though not real-time)	Broken ✗ ✗ Upscale on very large images (>4096px) fails silently ✗ No format selection — always outputs JPEG with fixed quality ✗ Upscale result sometimes mislinked to wrong canvas node ✗ Memory not freed after upscale — repeated upscales cause slowdown	Missing ■ ■ Side-by-side comparison view (before/after slider) ■ Batch upscale (upscale all nodes in canvas) ■ Format selection: JPEG quality, PNG lossless, WebP ■ AI-based upscaler selection (Real-ESRGAN vs Clarity vs others) ■ Partial upscale (upscale only a region via mask)
---	--	---

2.3 Variations

■ Variations	6.0/10	PARTIAL
Generates multiple variations of a selected image, either with different seeds (FLUX) or via img2img variation (Nano Banana 2). The critical known issue is that Nano Banana 2 does not support seed parameters (autoregressive architecture), so seed-based variation attempts produce nearly identical outputs rather than true variations. This is a fundamental model limitation that requires different UI guidance for each model.		
Working ✓ ✓ FLUX seed-based variations work correctly — distinct outputs per seed ✓ Variation strength slider correctly modifies output diversity ✓ Variations correctly added as sibling nodes on canvas ✓ Variation node links back to parent for traceability	Broken ✗ ✗ Nano Banana 2 variation with seed produces identical outputs (seed ignored) ✗ No UI warning that Nano Banana 2 does not support seeds ✗ Variation count defaults to 4 — no option to change per-session ✗ Variations sometimes overwrite instead of adding new nodes	Missing ■ ■ UI model capability badges: 'Supports seeds: No' for Nano Banana 2 ■ Variation grid view: 2x2 or 3x3 preview of all variations ■ Variation interpolation: smooth morphing between two images ■ Variation history: track which variation seed was chosen ■ Auto-variation: automatically generate 4 variations after each render

3. Data & Organization Features

History and Library — how users find, organize, and reuse their work

3.1 History

■ History	5.5/10	PARTIAL
The History page shows all past generation records with thumbnails, prompts, model info, and timestamps. It is the second most-used feature after Generate. The critical functional issue is no pagination — all records load on mount. The most-requested missing feature is the ability to click a history item and re-load it directly into a new canvas node for iteration.		

Working ✓ ✓ History records saved correctly after each generation ✓ Thumbnail preview displayed for each record ✓ Timestamp, model, and prompt metadata shown ✓ Delete individual record works ✓ Basic text search filters visible records	Broken ✗ ✗ All records loaded into memory on page mount — no pagination ✗ Search is client-side linear scan — slow with 100+ records ✗ Thumbnail generation sometimes fails, leaving blank preview ✗ Delete confirmation dialog sometimes fails to close ✗ Sort by date ascending/descending sometimes reversed	Missing ■ ■ Cursor-based pagination (load 20 at a time, infinite scroll) ■ 'Re-use' button: load history item into new canvas node ■ Filter by model (FLUX only, Nano Banana 2 only) ■ Filter by date range ■ Bulk select and delete ■ Export history to CSV/JSON ■ Favorite / star records for quick access
--	---	--

3.2 Library

■ Library	5.0/10	PARTIAL
The Library stores saved images, project assets, and reference materials. It is the least-developed major feature in the platform. Currently it functions as a flat image grid with no organization tools. For architectural professionals who manage dozens of projects simultaneously, the library needs collections, tagging, and project linking to be genuinely useful.		
Working ✓ ✓ Image upload and storage via Supabase Storage works ✓ Grid view displays library images correctly ✓ Image preview on click works ✓ Delete image from library works	Broken ✗ ✗ No folder or collection structure — all images in flat list ✗ Upload progress bar often stalls at 99% ✗ Large images (>10MB) fail to upload without clear error ✗ No drag-and-drop from Library directly to Canvas	Missing ■ ■ Collections / folders for project organization ■ Tags and labels for images ■ Smart folders: 'All FLUX renders', 'This week', 'Starred' ■ Drag-and-drop from Library to Canvas as source node ■ Reference image import from URL ■ Batch upload with progress ■ Image metadata view (model, prompt, seed used to generate) ■ Search by prompt text within library

4. Canvas & Workflow Features

4.1 Canvas — Node-Based Workflow

■ Canvas / Node-Based Workflow	6.5/10	PARTIAL
The canvas is the central UI of Anarchy AI — a node-based workflow editor where architects connect image sources, AI models, and processing steps. Currently in migration from ReactFlow to LiteGraph.js, the canvas shows the intended 240×320px card nodes with image previews and red circular connectors — but 40% of the migration is incomplete, causing hybrid state instability. When stable, the canvas is genuinely compelling.		

Working ✓ ✓ Node creation and deletion works in both ReactFlow and LiteGraph modes ✓ Basic connections between nodes (source → render → upscale) ✓ Viewport pan and zoom works ✓ Node position auto-save via Supabase ✓ Canvas background grid renders correctly ✓ 3ds Max bridge correctly injects screenshot as source node	Broken ✗ ✗ LiteGraph migration 40% complete — some node types use old ReactFlow API ✗ GhostNode type routing switches incorrectly between LiteGraph and ReactFlow ✗ Undo/redo history sometimes desynchronizes with canvas state ✗ Copy-paste of nodes loses connection metadata ✗ Right-click context menu occasionally appears off-screen	Missing ■ ■ Node grouping / framing (group related nodes visually) ■ Canvas minimap for large workflows ■ Node search / spotlight (Cmd+K to find node type) ■ Workflow templates: starter layouts for common architect tasks ■ Export workflow as .ana file format ■ Node comment/annotation cards ■ Auto-layout (arrange nodes automatically by data flow direction) ■ Collapse/expand complex nodes to save canvas space
---	---	--

5. Platform Features

Billing, Authentication, Export, and Notifications

5.1 Billing & Credits

■ Billing & Credits	6.8/10	PARTIAL
Billing is handled via Stripe with a credit-based consumption model. The integration is mostly complete — users can purchase credits and credits are deducted per generation. The main gap is incomplete Stripe webhook verification (important for security) and the lack of a pre-flight credit check before expensive operations.		
Working ✓ ✓ Stripe payment flow for credit purchase works ✓ Credit balance shown in UI and deducted per generation ✓ Billing history / invoice page accessible ✓ Low credit warning notification shown ✓ Free tier credit allocation on signup works	Broken ✗ ✗ Stripe webhook signature verification incomplete — replay attack risk ✗ Credit deduction sometimes fails silently — generation succeeds but credits not deducted ✗ Credit balance can go negative on rapid concurrent generations ✗ Refund flow not implemented — manual process required	Missing ■ ■ Pre-flight: check sufficient credits before submitting generation ■ Credit cost display per model before generation ■ Subscription tier (monthly credits instead of pay-per-use) ■ Team billing: shared credit pool for small firms ■ Usage analytics: credits spent per project/day/model ■ ZainCash integration for Iraqi market (IQD-denominated)

5.2 Authentication

■ Authentication	7.5/10	WORKING
Authentication is the strongest platform feature. Supabase Auth provides email/password login, password reset, and session persistence. The implementation is clean and reliable. Main improvements needed are MFA support and Google social login for professional users.		

Working ✓ ✓ Email/password sign-up and login works correctly ✓ Password reset via email works ✓ Session persistence across app restarts (JWT stored correctly) ✓ Auth-gated routes redirect correctly to login ✓ DEV_MODE bypass for development (properly guarded) ✓ Sign out correctly clears all session data	Broken ✗ ✗ Email verification flow occasionally sends to spam ✗ Session refresh fails after 7-day token expiry — silent logout	Missing ■ ■ Multi-Factor Authentication (MFA / TOTP) ■ Google OAuth social login ■ SSO for enterprise/firm accounts ■ Account deletion with data export ■ 'Remember this device' option
--	--	---

5.3 Export

■ Export		6.0/10	PARTIAL
Export handles image download, PDF generation, and DXF vector output. Image and PDF export work for basic cases. The DXF pipeline (Python FastAPI on port 7430) is the most fragile component — it depends on a Python subprocess that can crash silently, leaving the user with no feedback.			
Working ✓ ✓ Single image export (JPEG) works correctly ✓ PDF export with watermark works for single-image exports ✓ Export triggers correctly from canvas toolbar ✓ Watermark applied correctly to exported images	Broken ✗ ✗ DXF export fails silently if Python subprocess not running ✗ PDF export freezes UI for large canvases (>50 nodes) ✗ Export filename uses generic timestamp — not project name ✗ Multi-image PDF (one per canvas node) occasionally skips nodes	Missing ■ ■ Batch export: download all canvas outputs as ZIP ■ PNG export (currently JPEG only) ■ WebP export for web use cases ■ DXF export health check with auto-restart on failure ■ Export progress bar with estimated time ■ Export preset profiles (print quality, web quality, client presentation) ■ Direct upload to cloud storage (Google Drive, Dropbox)	

5.4 Notifications

■ Notifications		7.0/10	WORKING
The notification system using Zustand + toast components is the second-best implemented feature after Authentication. Generation complete, billing alerts, and error messages all trigger correctly. The main UX issue is notification flooding — a failed batch of 5 generations creates 5 identical error toasts.			

Working ✓ ✓ Generation complete toast triggers reliably ✓ Error notifications show for failed generations ✓ Low credit warning notification works ✓ Toast auto-dismiss after configurable timeout ✓ Notification type icons distinguish success/error/warning	Broken ✗ ✗ Duplicate notifications for repeated errors — no deduplication ✗ Notifications obscure canvas controls if many stack up ✗ Notification positioning inconsistent between pages	Missing ■ ■ Notification center (bell icon) with history of past 50 alerts ■ Deduplication: group identical messages with count badge ■ Desktop OS notifications for background generation complete ■ Notification preferences: mute certain notification types ■ In-app notification for new platform features / changelogs
---	--	--

6. Expected Bugs & Failure Modes

Predicted failure scenarios based on code analysis

The following bugs are predicted with high confidence based on the code patterns identified in the audit. They may not all be reproducible in a simple test session but will surface under real-world usage conditions.

ID	Severity	Bug Name	Scenario	Fix
BUG-001	CRITICAL	Canvas White Screen	Open a canvas with 50+ nodes after LiteGraph migration — hybrid state causes render error with no Error Boundary to catch it	Add Error Boundary around canvas; complete LiteGraph migration
BUG-002	CRITICAL	Credit Overdraft	Generate 6 images rapidly before credit deduction confirms — balance goes negative; generations may succeed without payment	Add pre-flight credit check; use atomic Supabase transaction for deduction
BUG-003	HIGH	Silent DXF Failure	Export to DXF when Python process not started — UI shows loading forever with no timeout	Add DXF bridge health check + 10s timeout with clear error message
BUG-004	HIGH	History Memory Leak	Open History page with 500+ records — browser process RAM exceeds 2GB, UI becomes unresponsive	Implement cursor pagination; load max 20 records at a time
BUG-005	HIGH	Variation Confusion	Run 4 variations on Nano Banana 2 — all 4 look identical because seed is ignored; user thinks feature is broken	Show model capability badge 'Seeds: Not Supported' for Nano Banana 2
BUG-006	HIGH	Double Generation	Click Generate, wait 3s, click again — two generations fire; both deduct credits; second overwrites first result	Disable Generate button during active job; use loading state from store
BUG-007	HIGH	Stale Auth Session	Leave app open 7+ days — JWT expires; next generation fails with auth error but user sees generic error	Implement automatic token refresh 24h before expiry
BUG-008	MEDIUM	Lost Generation on Network Drop	Generation running, network drops for 5s, returns — result is gone, credits deducted, no retry	Add Replicate webhook callback as fallback for polling-based result fetch
BUG-009	MEDIUM	Export Filename Collision	Export 3 images in same second — all get same timestamp filename — last one overwrites	Add UUID suffix to all export filenames

ID	Severity	Bug Name	Scenario	Fix
BUG-010	MEDIUM	Upscale Node Mismatch	Upscale on canvas with 4+ nodes — result sometimes linked to wrong parent node	Fix upscale result injection: pass explicit nodeId to upscale call

7. Feature Gap Analysis vs VizMaker

Where Anarchy AI leads, where it lags, and where to invest next

VizMaker is the primary competitor — a web-based architectural visualization AI tool. This comparison maps feature parity and identifies ANARCHY’s strategic advantages and the gaps that must be closed before the first public release.

Feature	ANARCHY AI	VizMaker	Verdict
Text-to-image generation	✓ Full	✓ Full	Parity
img2img (image + prompt)	✓ Full	✓ Full	Parity
Multiple AI models	✓ FLUX+NB2	✗ Single	ANARCHY leads
Node-based canvas workflow	✓ Full	✗ None	ANARCHY leads
3ds Max native bridge	✓ Full	✗ None	ANARCHY leads
Revit integration	■ Planned	✗ None	ANARCHY leads (when complete)
DXF vector export	✓ Partial	✗ None	ANARCHY leads
Generation history	✓ Partial	✓ Full	VizMaker better (pagination)
Image library / organization	✓ Partial	✓ Full	VizMaker better (collections)
Upscale	✓ Full	✓ Full	Parity
Variations / seeds	✓ Partial	✓ Full	VizMaker better (UI clarity)
Batch generation	✗ Missing	✓ Full	VizMaker wins
Real-time progress	✗ Fake	✓ Real	VizMaker wins
Generation cancel	✗ Missing	✓ Full	VizMaker wins
Collaboration / sharing	✗ Missing	✓ Partial	VizMaker wins
Offline mode	✗ Missing	✗ Missing	Neither
Mobile app	✗ Missing	✗ Missing	Neither
Style presets library	✓ Basic	✓ Full	VizMaker better
Prompt builder assistant	✗ Missing	✓ Partial	VizMaker leads
Client presentation mode	✗ Missing	✓ Partial	VizMaker leads
Local AI (Ollama)	■ Planned	✗ None	ANARCHY leads (when complete)

Feature	ANARCHY AI	VizMaker	Verdict
ZainCash / local payment	■ Planned	✗ None	ANARCHY leads (Iraqi market)

8. Functional Improvement Roadmap

Prioritized plan to reach functional parity and beyond

Sprint 1 — Fix Critical Broken Features

- Add generate button loading state — prevent double-submit
- Add pre-flight credit check before every generation
- Add Error Boundary around canvas — prevent white screen
- Fix Stripe webhook signature verification
- Add DXF bridge health check with 10s timeout
- Show Nano Banana 2 capability badge: 'Seeds: Not Supported'

Sprint 2 — History & Library Overhaul

- Implement cursor-based pagination in History (load 20 at a time)
- Move History search to Supabase full-text search
- Add 'Re-use in Canvas' button to history records
- Add Library collections / folders
- Enable drag-and-drop from Library to Canvas
- Add bulk select and delete in History

Sprint 3 — Generation Quality Improvements

- Add real-time Replicate webhook progress (replace fake timer)
- Add cancel button for running generation
- Implement generation queue with queue position indicator
- Add batch generation: 4 seeds in one click
- Add negative prompt field
- Show generation cost per model before submit

Sprint 4 — Canvas & Export Polish

- Complete LiteGraph migration — remove ReactFlow entirely
- Add canvas minimap
- Add node grouping / frame feature
- Add batch export (ZIP of all canvas outputs)
- Add PNG and WebP export formats
- Add export progress bar
- Add workflow templates (3 starter canvas layouts)

End of File 3 / 8

This concludes the Functional Audit. Continue with **File 4 — Performance Audit** for detailed analysis of memory leaks, re-render storms, bundle size optimization, canvas performance, and the top 20 performance issues with fix estimates.